

Warball — Technical Document

This document describes the work implemented so far: the Java/Spring Boot API, the MySQL schema behind it, the JSON player generator, and the Angular 21 frontend. It is written as a map of *what exists*, *why it exists*, and *what each piece does*.

Lore and sport design live in `warball_documentation.md` (and related notes). This file is the engineering counterpart.

1. What we are building

Warball is a browser-based, Blaseball-inspired sports sim set in Earisia. The long-term loop is:

- 1. Persist teams and players in MySQL.
- 2. Generate and edit rosters from an admin UI.
- 3. Run autonomous match ticks on the backend.
- 4. Stream those ticks to the public frontend (planned SSE + MongoDB).

What is live today is the directory half of that loop: teams with kit colors, player rosters, a procedural generator, public player cards, and an unauthenticated admin CRUD surface. Simulation, live feeds, and auth are not built yet.

We split the stack instead of a single app so the simulation engine can later tick without the UI, and so the Angular client can stay a thin REST consumer.

Layer	Choice	Why
Backend	Java 21, Spring Boot 3.5, executable JAR	Familiar Spring REST + JPA; deploy with <code>java -jar</code> , no standalone Tomcat
Relational DB	MariaDB/MySQL <code>warball_db</code> on port 3307	Durable teams/players; Hibernate <code>ddl-auto=update</code> during development
Documents (unused)	MongoDB <code>warball_live</code>	Reserved for match state and event logs; not read or written yet
Frontend	Angular 21, standalone, zoneless	Typed models matching the API; <code>HttpClient</code> + <code>async</code> pipe
Styling	Tailwind v4 + custom CSS variables	Blaseball-adjacent black/gold theme without a component library

2. Repository layout

```
warball/  
├── backend/                               Spring Boot API  
│   └── src/main/java/com/warball/backend/
```

├── BackendApplication.java	
├── controllers/	HTTP adapters
├── entities/	JPA tables
├── embeddables/	JSON-mapped value objects
├── repositories/	Spring Data JPA
├── services/	business rules
├── src/main/resources/	
│ ├── application.properties	
│ └── generator/	JSON catalogs for PlayerGeneratorService
├── frontend/warball/	Angular 21 app
│ └── src/app/	
│ ├── components/	public + admin screens
│ ├── models/	TypeScript mirrors of Java types
│ ├── services/	HttpClient wrappers
│ └── utils/country-flags.ts	
├── public/	logo + country flags
└── docs/	this file, screenshots, GIFs

Request flow for a typical public page:

Browser → Angular component → TeamService / PlayerService → HttpClient → /api/v1 → RestController → Service / Repository → MySQL

Admin Generate skips the manual form and hits `POST /teams/{id}/players/generate`, which runs `PlayerGeneratorService.generateForTeam`.

3. Backend

3.1 Configuration (`application.properties`)

Setting	Value	Why
<code>server.port</code>	<code>8080</code>	Local API; Angular talks to this host
<code>spring.datasource.url</code>	<code>jdbc:mysql://localhost:3307/warball_db</code>	XAMPP MariaDB is on 3307 so it does not collide with MySQL80 on 3306
<code>spring.datasource.username/password</code>	<code>root / empty</code>	Local XAMPP default

Setting	Value	Why
<code>spring.jpa.hibernate.ddl-auto</code>	<code>update</code>	Add columns (age, sex, hex colors, JSON blobs) without wiping rows
<code>hibernate.dialect</code>	<code>MariaDBDialect</code>	Matches the 3307 engine
<code>spring.data.mongodb.uri</code>	<code>mongodb://localhost:27017/warball_live</code>	Placeholder only

`BackendApplication` is a stock `@SpringBootApplication`. Component scan picks up controllers, services, and repositories under `com.warball.backend`.

3.2 Entity: `Team`

File: `entities/Team.java`

Teams are the top-level directory object. They exist so a roster has a parent, kit colors have an owner, and the public `/teams` page has something to list.

Field	Column	Why
<code>id</code>	<code>ID</code>	Surrogate PK; used in every nested URL
<code>teamName</code>	<code>TEAM_NAME</code>	Display name ("Galonian Gladiators")
<code>country</code>	<code>COUNTRY</code>	Nation in the Earisia setting; also the key for flag art
<code>province</code>	<code>PROVINCE</code>	Optional regional flavour; nullable
<code>homePrimaryHex / homeSecondaryHex</code>	<code>HOME_*</code>	Home kit fill + trim for the player-card SVG
<code>awayPrimaryHex / awaySecondaryHex</code>	<code>AWAY_*</code>	Away kit; stored now so we do not migrate later

Lombok `@Data` generates getters/setters. Four hex columns were added as a small, UI-unblocking step before the generator: without them the kit shirt would have nothing to paint.

There is **no** `OneToMany` players collection on `Team`. Rosters are loaded with `PlayerRepository.findByTeam_Id` so listing teams does not hydrate every player.

3.3 Entity: **Player**

File: `entities/Player.java`

Players are the simulation unit. Scalars that we filter/sort on are real columns. Nested, schemaless blobs are JSON columns so attributes can grow without an **ALTER** per skill.

```
@Table(uniqueConstraints = { TEAM_ID, SHIRT_NUMBER })
```

Shirt numbers are unique **per team**, not globally. Two Gladiators cannot both wear #4; a Flameweaver can.

Field	Column	Why
<code>id</code>	<code>ID</code>	DB PK; Angular routes use this (<code>/players/:playerId/details</code>)
<code>playerId</code>	<code>PLAYER_ID</code>	Human-readable code from the generator, e.g. <code>p_5793_dwa</code>
<code>firstName / lastName</code>	<code>FIRST_NAME / LAST_NAME</code>	Identity on roster rows and kit print
<code>age</code>	<code>AGE</code>	Scalar for display and later aging; generator rolls 18–40
<code>race</code>	<code>RACE</code>	Label from <code>races.json</code> ("Human"), not the JSON key
<code>subRace</code>	<code>SUB_RACE</code>	Label from <code>subraces.json</code> for that race key
<code>sex</code>	<code>SEX</code>	"Male" / "Female"; drives first-name pool
<code>position</code>	<code>PLAYER_POSITION</code>	One of Castle Keeper / Defender / Attacker
<code>team</code>	<code>TEAM_ID</code>	@ManyToOne — a player belongs to one club
<code>attributes</code>	<code>ATTRIBUTES</code> (JSON)	Nested physical/technical/mystical/vibe dice
<code>traits</code>	<code>TRAITS</code> (JSON)	Permanent catalog traits; not generated yet
<code>statusEffects</code>	<code>STATUS_EFFECTS</code> (JSON)	Match-time modifiers; empty at create
<code>hobbies</code>	<code>HOBBIES</code> (JSON array of strings)	Flavour text, e.g. "Dislikes hiking"
<code>shirtNumber</code>	<code>SHIRT_NUMBER</code>	1–99, unique on the team

JSON columns use Hibernate `@JdbcTypeCode(SqlTypes.JSON)`. That stores a document inside MySQL without Mongo, and Jackson serializes the same shape to the Angular client.

Convention: `race` / `subRace` persist **labels**. Catalog lookup uses lowercase **keys** (`human`, `dwarf`). Mixing those two caused bugs early (`Map.get(Player)` vs `race.key()`); the generator now looks up by `RaceEntry.key()`.

3.4 Embeddables (JSON value objects)

These are **not** `@Embeddable` JPA components. They are POJOs that Jackson writes into JSON columns. They live in `embeddables/` because they are owned by `Player` and have no table of their own.

PlayerAttributes

Four groups, so the future sim can weight "kick the ball" vs "ward a curse" vs "trash talk" separately.

- `physicalAttributes` → `PhysicalAttributes`
- `technicalAttributes` → `TechnicalAttributes`
- `mysticalAttributes` → `MysticalAttributes`
- `vibeAttributes` → `VibeAttributes`

PhysicalAttributes

Field	Intent
<code>heft</code>	Size / presence
<code>slipperiness</code>	Hard to pin
<code>stickyFingers</code>	Holding the ball
<code>paranoia</code>	Awareness / jumping at shadows
<code>bulwark</code>	Blocking

TechnicalAttributes

Field	Intent
<code>siege</code>	Power toward the castle
<code>trajectory</code>	Shot / pass flight
<code>threading</code>	Finding lanes
<code>magnetism</code>	Drawing the ball
<code>castleThirst</code>	Urge to attack the goal

MysticalAttributes

Field	Intent
<code>arcaneSpark</code>	Magic output
<code>ward</code>	Magical defence
<code>restraint</code>	Not exploding the pitch

VibeAttributes

Field	Intent
<code>trashTalk</code>	Mouth
<code>moxie</code>	Nerve
<code>dramaticFlair</code>	Theatre

All dice are `double` in **0.0–3.0** (see `roll()`). They are **not** sampled from JSON pools: attributes are random each generate, names/races are catalog-driven.

Trait

```
traitId, name, type
```

Permanent generated personality/mechanics. The class exists so the JSON column and the Angular model are ready. `generateForTeam` currently sets `traits` to `null`. **Do not purge the players table until traits generate.**

StatusEffect

Empty class. Status effects are match-time and should stay empty (or `[]`) at create. The player-details UI already has an empty-state list for them.

3.5 Repositories

Spring Data JPA supplies `save`, `findById`, `findAll` from `JpaRepository<T, Long>`.

TeamRepository — no custom methods. Listing and get-by-id are enough.

PlayerRepository

```
List<Player> findByTeam_Id(Long teamId);
```

Derived query: `findBy` + nested `team.id`. Used by roster endpoints and by `pickFreeShirtNumber` so the generator can see taken numbers. We did not load `Team.getPlayers()` to keep the team list cheap.

3.6 Services

Controllers should not contain create/update rules. Interfaces exist so the REST layer depends on a contract, not `*Impl`.

TeamService / TeamServiceImpl

Method	What it does	Why
<code>createTeam(Team)</code>	<code>teamRepo.save(team)</code>	POST body becomes a row; Hibernate assigns <code>id</code>
<code>updateTeam(Long, Team)</code>	Load existing, copy name/country/province/four hexes, save	Avoids replacing the entity and dropping fields the form omitted

`updateTeam` returns `Optional.empty()` when the id is missing so the controller can 404.

PlayerService / PlayerServiceImpl

Method	What it does	Why
<code>createPlayer(Player)</code>	<code>save</code> after the controller has attached a real <code>Team</code>	Manual admin create
<code>updatePlayer(Long, Player)</code>	Copy scalars + JSON blobs onto the existing row	Editing first name must not wipe attributes

Copied on update: `playerId`, names, `age`, `team`, race/subrace, `position`, `attributes`, `traits`, `statusEffects`, `hobbies`, `shirtNumber`.

Not copied: `sex`. The admin form does not edit sex, and `updatePlayer` has no `setSex`. Generated sex survives only because the form sends `loadedPlayer.sex` on create payloads in Angular; a PUT that omitted it would still leave the DB column unchanged (good), but a future form field needs a setter here.

JSON preserve on edit is intentional: the form only edits scalars. `loadedPlayer.attributes` is sent back so Hibernate does not write zeros.

PlayerGeneratorService

This is the procedural roster factory. Catalogs are JSON files under `classpath:generator/` so lore can be edited without a migration. DB tables for catalogs were deferred until we want an admin editor for names.

Lifecycle

1. Constructor injects `PlayerRepository`, `TeamRepository`, `ObjectMapper`.
2. `@PostConstruct loadCatalogs()` reads all JSON once at startup.
3. `generateForTeam(teamId)` builds one `Player` and `saves` it.

Inner records (Jackson-friendly, no extra files):

Record	Shape	Source file
<code>RaceEntry</code>	<code>key, label</code>	<code>races.json</code>
<code>SexedNames</code>	<code>male[], female[]</code>	<code>first-names.json</code>
<code>Hobbies</code>	<code>verbs[], hobbies[]</code>	<code>hobbies.json</code>

`subraces` and `lastNames` are `Map<String, List<String>>` keyed by race key. `positions` is a bare `List<String>`.

`loadCatalogs()`

Uses `ClassPathResource` + `objectMapper.readValue`. The same `ClassPathResource` variable is **reassigned**, not redeclared — a `resource` already-defined compile error taught that.

`pick(List<String>)`

Returns a random element, or `null` if the list is null/empty. Guard is required: `nextInt(0)` throws. Empty subrace lists (humans) produce `subRace = null` on purpose.

`roll()`

`random.nextInt(31) / 10.0` → 0.0, 0.1, ... 3.0. Attributes are dice, not catalog pools.

`pickFreeShirtNumber(Long teamId)`

Loads the current roster, collects taken numbers, builds free 1–99, picks one. Returns `null` only if the team somehow has 99 numbered players. Enforces the unique `(TEAM_ID, SHIRT_NUMBER)` constraint without catching a DB exception.

`generateForTeam(Long teamId)` — the public entry point

1. Load team or throw (unknown id).
2. New `Player`, set team.
3. Pick a `RaceEntry`; `playerId = "p_" + (1000–9999) + "_" + first three letters of the key`.
4. `race = label`; `subRace = pick` from that key's list.
5. Coin-flip sex; first name from `firstNames.get(key)` or "generic" fallback; male vs female lists compared with `.equals`, not `==`.
6. Last name from race key, else "generic".
7. Age `18 + nextInt(23)` (18–40).
8. Position from `positions.json`.
9. One hobby string: `verb + " " + hobby` (e.g. "Values studying magic").
10. Free shirt number.
11. `traits = null`, `statusEffects = null` (status should later be `[]`; traits wait for `traits.json`).
12. **New** `PhysicalAttributes` / `TechnicalAttributes` / `MysticalAttributes` / `VibeAttributes` / `PlayerAttributes` every call. Sharing one attribute object as a service field caused every generated player to receive the same numbers.
13. `save` and return the persisted player (201 from the controller).

3.7 REST controllers

Both controllers:

- `@RestController` + `@RequestMapping("/api/v1")`
- `@CrossOrigin(origins = "http://localhost:4200")` — Angular `ng serve`. Use `localhost`, not `127.0.0.1`, or the browser origin will not match.

TeamRestController

Method	Path	Behavior
GET	/teams	findAll, 200
GET	/teams/{id}	200 or 404
GET	/teams/{teamId}/players	404 if team missing; else roster list
POST	/teams	createTeam, 201
POST	/teams/{teamId}/players/generate	generateForTeam, 201
PUT	/teams/{id}	200 or 404

Roster lives under `/teams/{id}/players` rather than `/players?teamId=` so the URL matches the UI (`/teams/:id/players`). Generate is a POST on that collection because it creates a row.

`allTeams` and `findOneTeam` talk to the repository directly (no extra rules). Create/update go through `TeamService`. Generate goes through `PlayerGeneratorService`.

PlayerRestController

Method	Path	Behavior
GET	/players/{id}	One player including nested team, 200/404
POST	/players	Resolve <code>player.team.id</code> to a real Team, then <code>createPlayer</code> , 201
PUT	/players/{id}	Same team resolve, then <code>updatePlayer</code> , 200/404

Team resolve exists because JSON `{ "team": { "id": 1 } }` is a detached stub. Saving it as-is would either fail or insert a broken team. `teamRepository.findById` attaches the managed entity.

There is **no** GET `/players` (all players). Public and admin UIs always start from a team.

There is **no** DELETE. Cleanup is manual SQL until we want it.

3.8 Generator JSON catalogs

Path: `backend/src/main/resources/generator/`

File	Role
<code>races.json</code>	Ten <code>{ key, label }</code> races
<code>subraces.json</code>	Map of race key → list of subrace labels (human list may be empty)
<code>first-names.json</code>	Map of race key → <code>{ male, female }</code> ; plus "generic"
<code>last-names.json</code>	Map of race key → surnames; plus "generic"
<code>hobbies.json</code>	Global <code>{ verbs, hobbies }</code> — not per-race

File	Role
<code>positions.json</code>	<code>["Castle Keeper", "Defender", "Attacker"]</code>

Keys must match across files (`dwarf`, not `Dwarf`). Labels are what the UI shows. First names are sexed; last names are not.

Not present yet: `traits.json`. Next generator increment.

4. Frontend

4.1 Bootstrap

- `main.ts` bootstraps `App` with `appConfig`.
- `appConfig` provides the router, `HttpClient`, and browser error listeners. No `NgZone` `ChangeDetection` (Angular 21 zoneless).
- `App` is a shell: global `Header` + `<router-outlet>`.
- `index.html` loads **Oswald** (titles), **Lora** (body), **Share Tech Mono** (IDs) from Google Fonts.

`Header` is always visible: Home, Teams, Admin Mode (danger styling). Admin is not gated — auth waits for VPS deploy.

4.2 TypeScript models

`models/team/team.ts` mirrors `Team.java`, including the four hex fields as `string | null`.

`models/player/player.ts` mirrors `Player` plus the attribute interfaces. Extra care vs a first draft:

- No mongoose types (this is not Node).
- `paranoia` spelling matches Java.
- `statusEffects: StatusEffect[]` even though the Java class is empty.
- `shirtNumber: number | null` because generate can theoretically return null.
- Nested `team?: Team` because GET player includes the many-to-one.

These interfaces are the contract. If Java adds a field and Angular does not, the UI just ignores it; if Angular requires a field the API omits, templates need `?. / ?? []`.

4.3 HTTP services

Both are `providedIn: 'root'`. Base URL is hardcoded `http://localhost:8080/api/v1` for local CORS. That will need an environment file before VPS.

TeamService

Method	HTTP	Why strip <code>id</code> ?
<code>getTeams()</code>	GET <code>/teams</code>	List pages
<code>getTeam(id)</code>	GET <code>/teams/{id}</code>	Titles, kit colors, forms

Method	HTTP	Why strip id ?
<code>createTeam(team)</code>	POST <code>/teams</code>	Destructure <code>{ id, ...payload }</code> so Hibernate generates the PK
<code>updateTeam(id, team)</code>	PUT <code>/teams/{id}</code>	Id is in the URL, not the body

PlayerService

Method	HTTP
<code>getRoster(teamId)</code>	GET <code>/teams/{teamId}/players</code>
<code>getPlayer(id)</code>	GET <code>/players/{id}</code>
<code>createPlayer(player)</code>	POST <code>/players</code> (id stripped)
<code>updatePlayer(id, player)</code>	PUT <code>/players/{id}</code> (id stripped)
<code>generatePlayer(teamId)</code>	POST <code>/teams/{teamId}/players/generate</code> with <code>null</code> body

`generatePlayer` is a POST with no payload: the team id is the only input.

4.4 Routes (`app.routes.ts`)

Path	Component	Audience
<code>''</code>	<code>HomeComponent</code>	Public landing
<code>teams</code>	<code>TeamListComponent</code>	National teams
<code>teams/:id/players</code>	<code>PlayerListComponent</code>	Roster
<code>teams/:id/players/:playerId/details</code>	<code>PlayerDetailsComponent</code>	Player card
<code>admin/teams</code>	<code>AdminTeamListComponent</code>	Team CRUD hub
<code>admin/teams/new</code>	<code>AdminTeamFormComponent</code>	Create
<code>admin/teams/:id/edit</code>	<code>AdminTeamFormComponent</code>	Edit (same component, id present)
<code>admin/teams/:id/players</code>	<code>AdminPlayerListComponent</code>	Roster + Generate
<code>admin/teams/:id/players/new</code>	<code>AdminPlayerFormComponent</code>	Manual create
<code>admin/teams/:id/players/:playerId/edit</code>	<code>AdminPlayerFormComponent</code>	Edit

Public and admin share the same REST API. Admin routes are a second UI, not a second backend.

`:id` is always **team** id. `:playerId` is the numeric PK, not `playerId` string (`p_5793_dwa`).

4.5 Public components

HomeComponent

Landing copy, logo at `/images/warball-logo.png` (`public/images/`), CTA to `/teams`. New files under `public/` need an `ng serve` restart or they 404.

TeamListComponent

- `teams$ = this.teamService.getTeams()` at field init (one HTTP call per visit).
- Template `@for` + `async` pipe; `@empty` currently also looks like the loading state because `?? []` treats “not arrived yet” as empty.
- Each row links to `/teams/:id/players`.
- Flag image + two color swatches from `homePrimaryHex` / `homeSecondaryHex`.

flagSrc / **hideFlag**: `country-flags.ts` maps country string → `/flags/...`. If the image 404s, `(error)="hideFlag(country)"` records the country in a `signal<Set>` so Angular does not retry a broken URL in a loop.

PlayerListComponent

Reads `:id` from `ActivatedRoute`, loads `getTeam` (title + flag) and `getRoster`. Rows are `<a>` to details. Shirt number, name, position only — the card is the detail view.

PlayerDetailsComponent

Loads `getPlayer(playerId)` and `getTeam(teamId)`. The kit needs team hexes; `player.team` is usually populated by the API, but `teams$` is the dedicated source with `@let kitTeam = (teams$ | async) ?? player.team`.

The portrait is an **inline SVG jersey**, not a PNG:

- CSS variables `--kit-primary` / `--kit-secondary` from home hexes (fallbacks `#1a1a1a` / `#ffbe00`).
- Body path filled with primary, stroked with secondary; collar filled with secondary.
- Last name + shirt number overlaid (Football Manager-style print).

Panels: identity, details (including sex), hobbies, four skill groups, traits empty-state, status-effects empty-state. `@if (player.attributes; as attributes)` avoids crashing on older rows that still have `attributes: null`.

4.6 Admin components

Visual cue: `.wb-admin` + “Commissioner's office” banner and red-tinted danger buttons.

AdminTeamListComponent

Same `getTeams()` as public, plus **Add new team**, **Edit team**, **Go to roster** (admin roster, not public).

AdminTeamFormComponent

Reactive form (`FormBuilder`). `teamName` required. Color inputs are `type="color"` so hex values are always `#rrggbb`.

- No `:id` → create; POST then navigate to `/admin/teams`.

- With `:id` → GET, `patchValue`, PUT on submit.
- Failed GET navigates back to the list.

Empty hex strings become `null` so we do not store "".

AdminPlayerListComponent

Roster plus:

- **Add new player** → manual form.
- **Generate player** → `generatePlayer(teamId)`. On success it **reassigns** `this.players$` to a new `getRoster` observable. Navigating to the same URL does not refresh; replacing the observable does.
- **Edit player** per row.

`generateNewPlayer` copies `this.teamId` into a local `const` because the class field is `number | null` and is not narrowed inside `subscribe`.

AdminPlayerFormComponent

Scalar form only. JSON blobs are preserved from `loadedPlayer` (or `emptyAttributes()` / `[]` on create).

Hobbies are a **comma-separated string** in the input, split/trim/filter on submit, because a `List<string>` is awkward in a single text box.

`sex` is not on the form; create/update send `loadedPlayer?.sex ?? ''`. Combined with the missing Java `setSex`, new manual players can be stored with empty sex until the generator (or a future field) sets it.

Cancel links to the admin roster (`/admin/teams/:id/players`).

4.7 Flags and static assets

`utils/country-flags.ts`:

Country string (must match <code>Team.country</code>)	Asset
Galonian Empire	<code>/flags/galonian-empire.png</code>
Drakonar Empire	<code>/flags/drakonar-empire.svg</code>
Council of Zerathos	<code>/flags/council-of-zerathos.svg</code>

Lookup is exact (case-sensitive). "GALONIAN EMPIRE" would miss.

4.8 Theme

`styles.css` defines `--wb-gold`, cream text, diamond-grid background, shared `.wb-page`, `.wb-btn`, `.wb-row`, `.wb-form`. Component CSS files only add layout that is unique (kit SVG, player card width).

Tailwind is imported (`@import 'tailwindcss'`) but most of the look is the custom `wb-*` layer so the UI stays on-brand if Tailwind utilities are unused.

5. End-to-end: Generate → card

1. Commissioner opens `/admin/teams/1/players` and clicks **Generate player**.
2. `PlayerService.generatePlayer(1)` POSTs to `/api/v1/teams/1/players/generate`.
3. `PlayerGeneratorService` loads catalogs (already in memory), rolls identity + dice, saves MySQL.
4. Admin list refetches `GET /teams/1/players` and the new row appears (`#NN - Name - Position - Race`).
5. Public `/teams/1/players` shows the same row; click opens `/teams/1/players/{id}/details`.
6. Details paints the home kit from team hexes and lists attributes from the JSON column.

Manual create is the same persistence path without the generator (`POST /players` + form scalars).

6. Known gaps (do not treat as done)

- **Traits:** class + JSON column + UI list exist; generator does not roll them. Hold any `DELETE/TRUNCATE` of `players` until traits + empty `statusEffects` are verified.
 - `PlayerServiceImpl.updatePlayer` does not `setSex`.
 - **Auth:** admin is public on purpose until VPS.
 - **DELETE endpoints:** none.
 - **Simulation / SSE / MongoDB:** configured at most, unused.
 - **Loading vs empty:** `@empty` on lists fires while HTTP is in flight.
 - **CORS / API host:** hardcoded localhost; will break on a real domain.
 - **Shirt overlay on list rows:** kit is details-only.
-

7. Local run (engineering)

1. MariaDB/XAMPP on **3307**, database `warball_db`, empty root password (see `application.properties`).
2. `backend/: .\mvnw.cmd spring-boot:run` → `http://localhost:8080/api/v1/teams`.
3. `frontend/warball/: npm install && npx ng serve` → `http://localhost:4200`.
4. If `public/` files 404, restart `ng serve`.

Screenshots and GIFs of the current UI are in `docs/images/` and `docs/gifs/` (copied under `frontend/warball/docs/` for the frontend README).